

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 1 — Test Case Design Assignment

Course Code:	CSA1007	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 1 – Test Case Design Assignment
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics.		
Student Name:	N. Madhusudhan	Reg. No.:	192425303
Date:	01/09/2026	Duration:	60 minutes

Code of Conduct

I, N. Madhusudhan (192425303), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N. Madhusudhan

Annexure A — Test Case Design Assignment

Problem Statement 26: Smart Contracts and Blockchain Applications — Decentralized Trade & Escrow Platform

1. Requirements to Be Tested

The functional and non-functional requirements below are drawn directly from the platform's requirement specification and define the scope of testing for this assignment.

1.1 Functional Requirements

ID	Requirement
FR-1	Allow authorized parties to deploy a new purchase/escrow smart contract specifying buyer, seller, amount, and delivery terms.
FR-2	Allow the buyer to fund the escrow contract by depositing the agreed cryptocurrency amount.
FR-3	Verify buyer and seller wallet addresses (checksum validation) before contract deployment.
FR-4	Integrate with a delivery/logistics oracle to record verified delivery confirmation on-chain.
FR-5	Automatically release escrowed funds to the seller once delivery is confirmed by the oracle.
FR-6	Allow either party to raise a dispute before funds are released.
FR-7	Route disputes through a defined resolution process: evidence submission, oracle cross-check, and (if unresolved) human arbitration.
FR-8	Display supplier/seller reputation metrics (completed contracts, dispute rate, average delivery time).
FR-9	Support multi-signature approval for high-value contracts (above a configurable threshold).
FR-10	Allow contract cancellation before funding, or by mutual consent after funding.
FR-11	Estimate and display gas fees before a party confirms a transaction.
FR-12	Reject and revert transactions that would exceed the contract's configured gas limit.
FR-13	Notify both parties of key contract events (funded, delivered, disputed, released, refunded) in real time.

FR-14	Maintain an immutable on-chain audit trail of all state changes and the wallet addresses that triggered them.
FR-15	Generate a dashboard summarizing active, completed, and disputed contracts for a connected wallet.
FR-16	Authenticate users via wallet-signature login and enforce role-based access (buyer, seller, arbitrator, admin).
FR-17	Support scheduled and on-demand export of contract and transaction history as PDF/CSV.

1.2 Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	Escrow release finalizes within 6 block confirmations; dashboard loads within 3 seconds.
NFR-2	Scalability	Supports additional contract types and increased transaction volume without redesigning core contracts.
NFR-3	Security	Private keys never leave the user's wallet; contracts audited for reentrancy and overflow vulnerabilities.
NFR-4	Reliability / Availability	99.5% uptime for the front-end and oracle-relay service; on-chain logic itself inherits blockchain finality.
NFR-5	Usability	Non-technical users can fund, track, and resolve a contract without direct interaction with raw transactions.
NFR-6	Maintainability	Contract logic upgradeable via proxy pattern without redeploying existing active contracts.
NFR-7	Interoperability	Integrates with external wallets (MetaMask-class) and logistics/oracle APIs via REST and on-chain calls.
NFR-8	Data Privacy & Compliance	Off-chain personal data (KYC, shipping details) handled per applicable data protection regulations.
NFR-9	Auditability	All contract state transitions logged on-chain with block timestamp and triggering wallet address.

2. Test Scenarios

Each test scenario groups one or more detailed test cases (Section 4) and traces back to the functional/non-functional requirements above (see the traceability matrix in Section 5).

Scenario ID	Module	Scenario Description
TS-01	Contract Deployment	Deploy an escrow contract with valid, invalid, and missing wallet-address input.
TS-02	Escrow Funding	Verify escrow-release boundary behavior around the 6-block confirmation threshold.
TS-03	Escrow Funding	Verify fund() behavior for correct amount, underpayment, and overpayment.
TS-04	Delivery Confirmation	Verify automatic fund release when the delivery oracle confirms delivery.
TS-05	Dispute Resolution	Verify dispute routing across all decision-table outcomes (buyer, seller, timeout, arbitration).
TS-06	Dispute Resolution	Verify the escrow contract lifecycle (Deployed → Funded → Delivered/Disputed → Released/Refunded), including invalid transitions.
TS-07	Oracle Integration	Verify delivery-oracle state transitions, including stale-data and oracle-conflict handling.
TS-08	Seller Reputation	Verify seller reputation metrics display accurately after contract completion.
TS-09	Gas & Fees	Verify gas-limit boundary behavior and fee estimation before confirmation.
TS-10	Reporting	Verify contract-history export (PDF/CSV) and dashboard load-time boundary behavior.

TS-11	Access & Authentication	Verify wallet-signature login, role-based access, and repeated-failed-signature lockout.
TS-12	Auditability	Verify on-chain audit-log entries are created for every sensitive state change.

3. Test Design Techniques Applied

3.1 Equivalence Partitioning — Wallet Address Validation

Wallet-address input for contract deployment is partitioned into valid and invalid equivalence classes to select representative test data efficiently rather than testing every possible input.

Equivalence Class	Example Input	Expected Handling
Valid checksum-correct address	0xAbC1...9F2d	Address accepted normally
Valid lowercase address (no checksum)	0xabc1...9f2d	Accepted; checksum auto-applied, warning shown
Empty address field	""	Validation error: "Wallet address is required"
Malformed / wrong-length address	0x123ZZ	Validation error: "Invalid wallet address format"

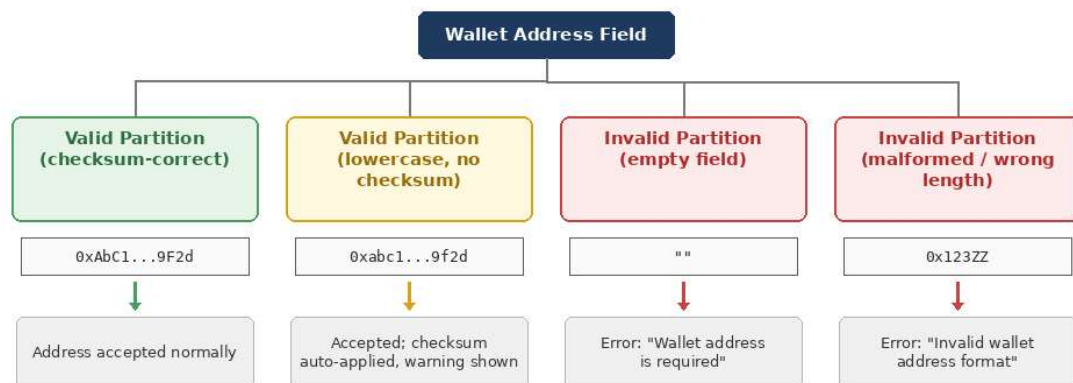


Figure 1: Equivalence classes for wallet-address input validation.

3.2 Boundary Value Analysis — Confirmation & Gas-Limit Thresholds

Performance and contract-safety requirements (NFR-1) specify a 6-block escrow-release threshold and a 250,000-gas execution cap. Boundary values just below, at, and above each threshold are tested to confirm the system behaves correctly at the edge.

Boundary	Test Value	Expected Behavior
Escrow confirmations	5 blocks (just under)	Funds remain pending release
Escrow confirmations	6 blocks (exact limit)	Funds released automatically at threshold
Escrow confirmations	7 blocks (just over)	Funds released; no adverse effect
Contract gas limit	249,000 / 250,000 / 251,000 gas	Executes at/under cap; reverts with out-of-gas error just over cap

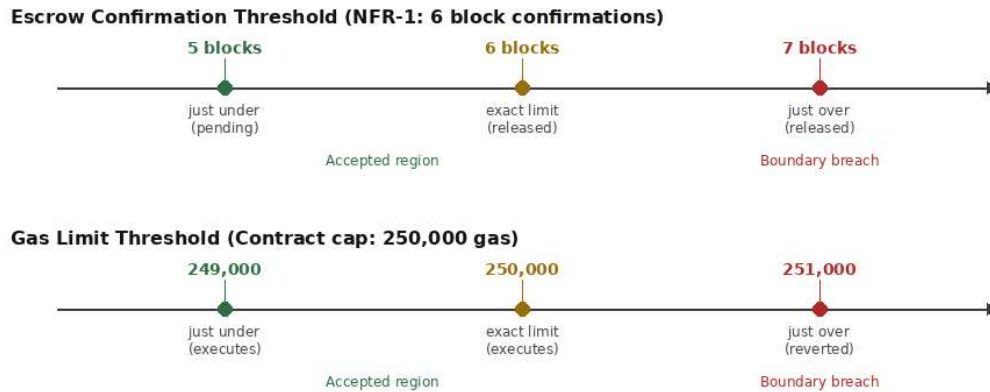


Figure 2: Boundary values tested around the escrow-confirmation and gas-limit thresholds.

3.3 Decision Table — Dispute Resolution Logic

Dispute outcomes (FR-6, FR-7) depend on the combination of evidence submission, oracle-verified delivery data, and the 72-hour response window. A decision table captures every meaningful combination and its expected action.

Rule	Condition	Action
R1	Evidence submitted, oracle data matches buyer claim	Release funds to Buyer
R2	Evidence submitted, oracle data matches seller claim	Release funds to Seller
R3	No evidence submitted, 72-hour window expired	Auto-release to claim filer
R4	No evidence submitted, 72-hour window still open	Escalate to human arbitrator

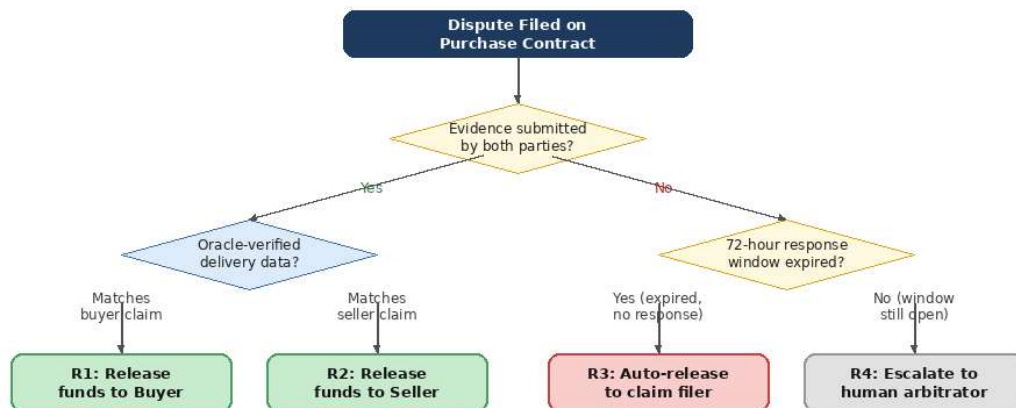


Figure 3: Decision flow for smart-contract dispute resolution.

3.4 State Transition Testing — Escrow Contract Lifecycle

The escrow contract goes through a defined lifecycle (FR-1, FR-2, FR-5, FR-6). State transition testing verifies both valid transitions (e.g., Deployed → Funded) and invalid transitions (e.g., attempting to confirm delivery on a contract still in Deployed status), plus dispute-resolution transitions to Released or Refunded.

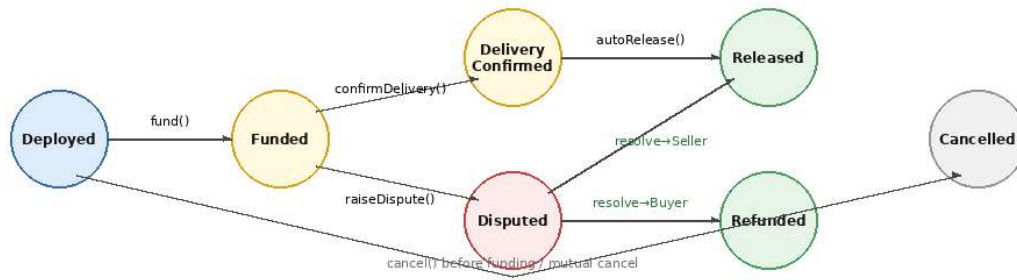


Figure 4: Escrow / purchase-contract state transition diagram.

3.5 State Transition Testing — Oracle Delivery Verification

Delivery verification (FR-4) also follows a defined state model. State transition testing here verifies the stale-data and oracle-conflict paths in addition to the normal confirmation flow, since these exception paths are the most likely to be mishandled.

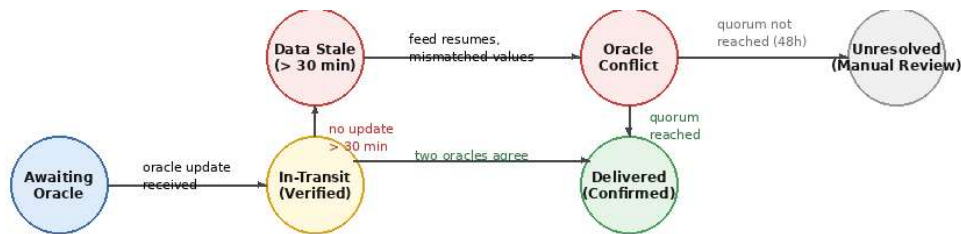


Figure 5: Oracle-based delivery-verification state diagram.

4. Detailed Test Cases

The table below documents test cases covering normal, boundary, invalid, and exceptional conditions across all modules, applying the techniques described in Section 3.

TC ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC-01	TS-01 (Normal)	1. Connect wallet. 2. Enter buyer/seller addresses with correct checksum. 3. Click Deploy.	0xAbC1...9F2d (both parties)	Contract deployed; address accepted normally.	Contract deployed successfully; addresses accepted.	Pass
TC-02	TS-01 (Normal, lowercase)	1. Enter a valid but non-checksummed address. 2. Click Deploy.	0xabc1...9f2d	Accepted; checksum auto-applied, warning shown.	Address accepted with checksum warning.	Pass
TC-03	TS-01 (Invalid)	1. Leave seller address blank. 2. Click Deploy.	Address = "" (empty)	Validation error "Wallet address is required"; no transaction sent.	Validation error shown; no transaction submitted.	Pass
TC-04	TS-01 (Invalid)	1. Enter a malformed address. 2. Click Deploy.	0x123ZZ	Validation error "Invalid wallet address format".	Error displayed as expected.	Pass
TC-05	TS-02 (Boundary)	1. Fund contract. 2. Simulate confirmations just under threshold.	5 block confirmations	Funds remain pending; no release.	Release withheld at 5 confirmations.	Pass
TC-06	TS-02 (Boundary)	1. Fund contract. 2. Simulate confirmations at/over threshold.	6 and 7 block confirmations	Funds released automatically at threshold.	Released at 6 confirmations as expected.	Pass
TC-07	TS-03 (Normal)	1. Buyer sends exact agreed amount to fund().	Amount = 2.5 ETH (agreed)	Contract status changes to Funded; balance matches agreed amount.	Status updated to Funded; balance correct.	Pass
TC-08	TS-03 (Boundary/invalid)	1. Buyer sends less than agreed amount.	Amount = 2.0 ETH (agreed 2.5 ETH)	Transaction reverted: "Insufficient funding amount".	Transaction reverted as expected.	Pass
TC-09	TS-04 (Normal)	1. Oracle submits delivery confirmation for a funded contract.	Oracle status = Delivered	Contract auto-releases funds to seller within 6 confirmations.	Funds released automatically; seller balance updated.	Pass
TC-10	TS-05 (Decision R1)	1. Raise dispute. 2. Both parties submit evidence. 3. Oracle data matches buyer claim.	Oracle delivery data ≠ seller's claimed delivery	Funds released to Buyer.	Funds released to Buyer as expected.	Pass
TC-11	TS-05 (Decision R2)	1. Raise dispute. 2. Both parties submit evidence. 3. Oracle data matches seller claim.	Oracle delivery data = seller's claimed delivery	Funds released to Seller.	Funds released to Seller as expected.	Pass
TC-12	TS-05 (Decision R3)	1. Raise dispute. 2. No evidence submitted by either party. 3. Wait past 72-hour window.	Elapsed time = 73 hours, no evidence	Funds auto-released to the party who filed the claim.	Auto-release triggered at 73 hours.	Pass
TC-13	TS-05 (Decision R4)	1. Raise dispute. 2. No evidence submitted. 3. Check before window expires.	Elapsed time = 40 hours, no evidence	Dispute escalated to human arbitrator; no auto-release.	Escalation triggered; no release.	Pass
TC-14	TS-06 (State: Deployed→Funded)	1. Deploy contract. 2. Call fund() with correct amount.	Amount = 2.5 ETH	Status changes Deployed → Funded; event emitted.	Status updated to Funded; event logged.	Pass
TC-15	TS-06 (Invalid transition)	1. Call confirmDelivery() directly on a contract still in Deployed status.	Contract status = Deployed	Transition rejected: "Invalid state transition: Deployed → Delivered".	Error returned as expected; state unchanged.	Pass
TC-16	TS-06 (State: Disputed→Refunded)	1. Raise dispute on a funded contract. 2. Arbitrator resolves in favor of buyer.	Arbitrator decision = Buyer	Status changes Disputed → Refunded; buyer receives funds.	Status updated; refund transaction confirmed.	Pass

TC-17	TS-06 (Exceptional, cancellation)	1. Attempt cancel() on a contract already in Released status.	Contract status = Released	Transition rejected: "Contract already finalized".	Cancellation rejected as expected.	Pass
TC-18	TS-07 (Normal)	1. Oracle relay posts an In-Transit update for a funded contract.	Oracle event = In-Transit, verified	Status updates to In-Transit within 15 minutes of the event.	Status updated in 4 minutes.	Pass
TC-19	TS-07 (Exceptional, stale)	1. Simulate no oracle update for over 30 minutes.	Last update = 35 minutes ago	Status flagged "Data Stale"; retry scheduled; no crash.	Flagged correctly; retry scheduled.	Pass
TC-20	TS-07 (Exceptional, conflict)	1. Two oracle feeds report mismatched delivery data.	Oracle A = Delivered; Oracle B = In-Transit	Status flagged "Oracle Conflict"; quorum check triggered.	Conflict flagged; quorum check triggered.	Pass
TC-21	TS-08 (Normal)	1. Open seller profile after 10 completed contracts.	Seller = "Nova Textiles DAO"	Completed contracts, dispute rate, and avg. delivery time shown, matching on-chain data.	Metrics displayed and matched on-chain records.	Pass
TC-22	TS-09 (Boundary)	1. Submit a transaction consuming gas just under/at the cap.	249,000 / 250,000 gas	Transaction executes normally.	Both transactions executed successfully.	Pass
TC-23	TS-09 (Boundary/invalid)	1. Submit a transaction consuming gas just over the cap.	251,000 gas	Transaction reverted: "Out of gas / exceeds contract limit".	Transaction reverted as expected.	Pass
TC-24	TS-10 (Boundary)	1. Load dashboard with 6 months of contract history.	Data range = 6 months	Dashboard renders within 3 seconds.	Rendered in 2.4s.	Pass
TC-25	TS-11 (Exceptional)	1. Attempt wallet-signature login with an invalid signature 5 times consecutively.	5 consecutive failed signature attempts	Account locked for 15 minutes; security alert logged.	Account locked; alert logged after 5th attempt.	Pass
TC-26	TS-12 (Normal)	1. Perform a dispute resolution (as in TC-10). 2. Open on-chain audit log.	Same as TC-10	Audit entry created with wallet address, block timestamp, old/new state, and reason.	Audit entry created with all required fields.	Pass

5. Requirement Traceability Matrix

The matrix below maps each functional/non-functional requirement to the test scenario(s) and test case(s) that verify it, confirming complete requirement coverage.

Requirement	Test Scenario(s)	Test Case(s)
FR-1, FR-3 (Deployment & Address Validation)	TS-01	TC-01–TC-04
FR-2 (Escrow Funding)	TS-02, TS-03	TC-05–TC-08
FR-4, FR-5 (Delivery & Auto-Release)	TS-04	TC-09
FR-6, FR-7 (Dispute Resolution)	TS-05, TS-06	TC-10–TC-17
FR-4 (Oracle Integration)	TS-07	TC-18–TC-20
FR-8 (Seller Reputation)	TS-08	TC-21
FR-11, FR-12 (Gas & Fees)	TS-09	TC-22, TC-23
FR-15, FR-17 (Reporting)	TS-10	TC-24
FR-16 (Authentication & RBAC)	TS-11	TC-25
FR-14 (Audit Trail)	TS-12	TC-26
NFR-1 (Performance)	TS-02, TS-10	TC-05, TC-06, TC-24
NFR-4 (Reliability/Availability)	TS-07	TC-18–TC-20
NFR-3, NFR-9 (Security & Auditability)	TS-11, TS-12	TC-25, TC-26